STEAMGEARS.EU · NYE SITES

Nye WordPress-sites

guide, metode og sammenligning

Skrevet: 8/11 2026 · **Gælder:** de næste sites vi starter, ikke steamgears.eu · **Kilder:** §15

Denne guide svarer på to ting: **hvad er den bedste og mest moderne måde at bygge et WordPress-site på i dag**, og **hvordan står den metode mod den vi selv bruger**. Den er skrevet, så et nyt site kan startes på under to uger uden at gentage de fejl steamgears har betalt for.

Den erstatter ikke deploy-playbook-2026-11-08.md, der handler om at publicere **en side** på et site der allerede kører. Denne handler om at **vælge og bygge sitet**.

---

0. Kort opsummering

Den moderne metode (Git- og staging-metoden) har fem dele, og de hænger sammen:

1. **Prototype i WordPress Playground** med en Blueprint-JSON i repoet — et helt site på sekunder, uden server, perfekt til demo og kundegodkendelse.
2. **Byg lokalt** med `wp-env`, DDEV eller WordPress Studio, med WP-CLI og PHP i samme version som drift.
3. **Block theme med theme.json** — ikke page builder. Designet er kode, ikke 900 indstillinger i en database.
4. **Git → staging → live.** Hostens eget Git-deploy eller en CI-pipeline over SSH; staging er en klon, live er beskyttet, og rollback er en tidligere release.
5. **Managed drift:** backup med testet restore, WAF/CDN, object cache, automatiske opdateringer der først rammer staging, og WP-CLI/SSH til fejlfinding.

Vores metode er Simply + wp-admin/MCP + et separat dokumentationsrepo. Den har **ingen staging, ingen Git-deploy af site-koden og ingen lokal reproduktion**, men til gengæld en måle- og værnkultur der er bedre end de fleste: hver side måles, hver commit bærer procenter, og hver fælde bliver et værn.

Konklusion i tre sætninger: til **nye sites** er Git- og staging-metoden det rigtige valg — den koster mere pr. måned og kræver SSH/Git-kompetence, men den fjerner præcis de tre svagheder der har kostet os dyrest (ingen staging, ingen rollback af kode, fejlfinding gennem en cache). Til **steamgears** skal vi ikke migrere for migrationens skyld; siden kører, er målt, og en flytning ville koste mere end den giver lige nu. Uanset hvad vi vælger, tager vi **værnene og dokumentationen** med — de er metoden, ikke hosten. Den nye arbejdsrækkefølge er: **mål → afgræns → prototype én kritisk brugerrejse → test med målgruppen → byg → kontrollér kvalitet → staging-accept → live → mål igen**. Vi skalerer ikke indhold eller features op, før den kritiske rejse virker.

Spørgsmål	Vores metode	Git- og staging-metoden
Kan en ændring fortrydes på sekunder?	Nej, ikke kode	Ja, forrige release
Kan en ændring testes før live?	Nej	Ja, på staging
Er site-koden versionsstyret?	Kun plugin'et	Ja, hele temaet og egen kode
Kan sitet genskabes lokalt?	Nej	Ja, med kommando
Koster det mere pr. måned?	Nej	Ja, typisk 2–10× shared
Kræver det mere af den der driver det?	Nej	Ja: Git, SSH, deploy

---

1. Den moderne metode — de seks lag

Lag	Værktøj i 2026	Hvorfor	Bevis
0. Beslut	indholdsmodel, slugs, redirects	slug-bytte efter live kræver 301	egen erfaring, deploy-playbook §3.0
1. Prototype	Playground + Blueprint-JSON	demo på sekunder, samme udgangspunkt hver gang	Playground Handbook, Blueprints Docs
2. Lokal	wp-env, DDEV, WordPress Studio	teste PHP/plugins før de rammer nogen	Theme Handbook: Tools and Setup
3. Tema	block theme + theme.json + patterns	design som kode, ingen builder-lås	Theme Handbook, 7.1 dev notes
4. Kode	Git-repo (tema, egen plugin, mu-plugins)	historik, review, rollback	Bedrock-dokumentation
5. Deploy	hostens Git-deploy eller CI over SSH	staging → live med et klik eller en merge	WP Pusher-hostmatrix 20/10 2026
6. Drift	backup + restore-øvelse, WAF, object cache, WP-CLI	fejl skal findes i logs, ikke i cachen	Kinsta/Raidboxes-dokumentation

1.1 Prototype: Playground er blevet en rigtig arbejdsflade

WordPress Playground kører WordPress i browseren eller i Node via PHP-WASM. I 2026 fik det en ny grænseflade til flere sites, en samlet **Playground Handbook** og en **Site Manager API**, der kan oprette, liste, ændre og slette Playground-sites fra kode — altså scriptede demoer og testmatricer uden en server. En Blueprint er en JSON-fil, der beskriver hele opsætningen: versioner, tema, plugins, indhold, indstillinger og login.

```
{
  "$schema": "https://playground.wordpress.net/blueprint-schema.json",
  "preferredVersions": { "php": "8.3", "wp": "latest" },
  "landingPage": "/wp-admin/",
  "steps": [
    { "step": "installTheme", "themeData": { "resource": "wordpress.org/themes", "slug":
"twentytwentyfive" },
      "options": { "activate": true } },
    { "step": "setSiteOptions", "options": { "blogname": "Nyt site" } },
    { "step": "login", "username": "admin", "password": "password" }
  ]
}
```

Start den med `npx @wp-playground/cli@latest server --blueprint=blueprint.json`. Fordi filen ligger i repoet, er en demo ikke længere noget nogen har "liggende på sin maskine" — den er reproducerbar.

Grænsen: Playground er til demo, test og prototyping. Det er ikke drift: intet persistent backend, ingen rigtige betalinger, ingen garanti for at et plugin med server-afhængigheder opfører sig som i produktion.

1.2 Lokal udvikling: vælg ét værktøj og hold fast

Værktøj	Bedst til	Bemærkning
wp-env	standard, CI-venligt	Docker; konfigureres i .wp-env.json
DDEV	teams der vil have mail, DB-værktøjer, HTTPS	Docker; ddev start
WordPress Studio	den der vil have en app og et klik	Automattic-værktøj, mindre CI-orienteret
Playground CLI	hurtig test af plugin/tema	ingen persistens uden mount

Vælg **ét** og skriv det i repoets README. Det vigtige er ikke værktøjet, men at PHP-versionen og

WordPress-versionen i det lokale miljø **matcher drift** — ellers tester man noget andet end det man udgiver.

1.3 Tema: block theme er standarden, og 7.1 gør den stærkere

WordPress 7.1 landede 19/8 2026. For et nyt site betyder det tre ting:

- **Editoren er altid iframed** — også for klassiske temaer og legacy meta boxes. Man skal ikke længere teste "med og uden iframe".
- **Responsive styles i theme.json** under `@mobile` og `@tablet`, og temaet kan selv sætte `settings.viewport` (standard 480 px og 782 px). Mobile CSS er ikke længere et hack i et child theme.
- **Pseudo-states** (`:hover`, `:focus`, `:focus-visible`, `:active`) kan defineres i `theme.json` for Button og Navigation Link.

WooCommerce kræver et **block theme** for at give adgang til hele Site Editor, inklusive produkt- og katalogskabeloner, og kurv/kasse-blokkene er standard for butikker startet efter 8.3. Order storage er **HPOS** som standard for nye installationer siden 8.2 — det skal man ikke slå til, men man skal kontrollere at hvert plugin erklærer sig kompatibelt.

Page builders er ikke forbudt, men de er et valg med en regning: designet ender i databasen, ikke i Git, og hver builder tilføjer sin egen HTML, CSS og JavaScript. Til et site med en fast designidentitet er `theme.json` + patterns både hurtigere og lettere at rulle tilbage.

1.4 Deploy: to ærlige veje

Vej	Sådan	Styrke	Pris
Hostens Git-deploy	git push til et remote, hosten tjekker ud	intet CI at vedligeholde	afhængig af host; nogle låser Test/Live
CI over SSH	GitHub Actions → rsync/SSH → staging	samme pipeline uanset host	kræver nøgler og en pipeline

Konkret, målt 20/10 2026: **WP Engine** har GitPush og en officiel GitHub Action (men `wp-config.php` og `wp-content/uploads` kan ikke pushes), **Pressable** har GitHub-integration, **WordPress.com** har GitHub Deployments på Business og Commerce, **Kinsta** har Git installeret men **ingen push-to-deploy** (man puller over SSH eller kører CI), **Pantheon** kører Dev → Test → Live hvor Test og Live er låste, og **Cloudways** har git auto-deploy via webhooks. Fælles for dem alle: **databasen er ikke i Git.** Indhold, ordrer og indstillinger lever i databasen, og en deploy flytter kun kode.

Derfor er reglen: kode deployes automatisk, **indhold** migreres bevidst. En `staging -> live` -synkronisering af databasen kan overskrive kundeordrer, hvis den køres uden at tænke.

1.5 Bedrock og Composer: kun når man har brug for det

Bedrock (Roots) lægger WordPress i en `web/wp`-mappe, lader Composer styre core og plugins, og bruger `.env` til miljøvariabler. Core og plugins er **ikke** i Git; kun egen kode og `composer.json`/`composer.lock`. Det giver en ren historik og reproducerbare versioner.

Det kræver til gengæld SSH eller en host der accepterer strukturen, og det kræver at man vælger hvem der må opdatere core: Composer eller WordPress' auto-opdatering. **Beslutningen skal træffes før første deploy**, ellers slås de to systemer om de samme filer.

Tommelfingerregel: brug Bedrock hvis I har mere end et par egne plugins eller flere udviklere. Ellers er et almindeligt repo med `wp-content/themes/<tema>` og `wp-content/mu-plugins/` nemmere at drive.

1.6 Drift: det der gør metoden værd at betale for

- **Backup + testet restore.** En backup der aldrig er gendannet, er en formodning. Gendan til staging hvert kvartal og skriv det ned.

- **Staging med noindex og password.** Ellers konkurrerer to kopier af sitet om de samme søgeord.
- **Opdateringer først på staging.** Plugin-opdateringer er den hyppigste årsag til hvide sider.
- **WAF og CDN foran**, men med en kendt måde at måle uden om cachen.
- **Logs og fejlsporing** uden for webroden, så en fatal fejl ikke kun findes i browseren.
- **WP-CLI og SSH**, så en fejl kan diagnosticeres i stedet for at gættes.

---

2. Vores metode, som den faktisk er

Det her er ikke en karikatur; det er den målte tilstand pr. 8/11 2026, og flere af delene er gode.

Faktum	Værdi	Kilde
Host	Simply, delt hosting	docs/agent-profil.md §1
Stack	WordPress 7.1.4, PHP 8.5.12, WooCommerce 11.2.2, 22 aktive plugins	docs/roadmap.md §2
Adgang	wp-admin + MCP (Novamira/AI Engine); ingen SSH/Git i driften	docs/agent-profil.md §4
Kode i Git	plugin-kilde, mu-plugins, CSS, dokumentation — ikke hele sitet	README.md
Deploy	GitHub gør zip'en tilgængelig; WordPress installerer den	plugins/steamgears-autopilot/README.md
Staging	findes ikke	docs/library/deploy-roadmap-2026-11-03.md
Rollback	kladde, revision, fil-backup, option, gammel zip fra git	deploy-playbook §7
Backup	fil- og DB-backup taget 23/10 og 27/10 2026	backup-2026-10-23/, backup-2026-10-27/
Måling	6 projekter med beregnede procenter, 148 historik-rækker	projektstatus.json
Værn	commit-besked, status-drift, fejlscan, PDF-render, smoke-tests	.github/workflows/

Det der er stærkt, og som skal med videre:

- **Dokumentation som hukommelse.** Hver fælde bliver et dokument med dato, bevis og en regel. Det er derfor denne guide kan skrives.
- **Måling før påstand.** Procenter beregnes, ikke gættes; 100 % kræver `verified: true`.
- **Automatiske værn** på commit-form, status-drift, zip-integritet og PDF-render.
- **Agent-adgang over MCP** gør gentagne opgaver scriptbare uden at give en agent SSH.
- **Billig drift.** Et lille site som dette skal ikke koste som en mellemstor virksomhed.

Det der koster, og som den nye metode løser:

Svaghed	Hvad den har kostet
Ingen staging	en plugin- eller indstillingsfejl rammer kunderne direkte
Ingen kode-Git for sitet	en ændring i wp-admin kan ikke diffes eller rulles tilbage
Cache foran PHP	en rettet fatal fejl så ud som en fortsat nedbrudt side
WAF og bot-filtre	målinger kan ramme en challenge i stedet for sitet
Plugin-mappe = ejerens klik	en release er ikke live før et menneske trykker
Ingen lokal reproduktion	fejl findes i produktion, ikke før

---

3. Sammenligning

Område	Vores metode	Git- og staging-metoden	Konsekvens
Hosting	delt hosting, billig	managed WP eller VPS med SSH	pris op, men også værktøjer
Adgang	wp-admin + MCP	SSH, WP-CLI, Git, MCP	fejl findes i logs
Site-kode	i databasen og filer	i Git	diff og review
Deploy	zip/klik	merge eller git push	sekunder vs. minutter
Staging	nej	klon med ét klik	test før live
Rollback	revision/fil/option	forrige release	minutter vs. manuel
Backup	manuel fil/DB	daglig + restore-øvelse	gendannelse er prøvet
Secrets	i options og wp-config	.env/host-secrets	rotation og lækage
Lokal reproduktion	nej	ja	fejl før produktion
Cache	W3TC i WordPress	edge/CDN + object cache	kræver purge-rutine
Sikkerhed	WAF hos host, 22 plugins	managed WAF + færre plugins	mindre angrebsflade
Ændringshastighed	høj for indhold, lav for kode	høj for begge	flere leverancer
Kompleksitet	lav	middel	kræver kompetence
Pris pr. måned	lav	2–10× (estimat)	skal vejes mod timer

Læs rækkerne som en prioritering, ikke en dom. Den største enkeltforbedring er **staging + Git-deploy**; den næststørste er **lokal reproduktion**. Resten er optimering.

4. Hvad »bedre« skal betyde — måltal

Et nyt site skal måles på det samme, uanset metode, så "bedre" ikke bliver en mening:

Måltal	Hvorfor	Baseline at slå
Deploy-frekvens	hvor ofte kan vi levere?	steamgears: content dagligt, kode ved release
Lead time	fra beslutning til live	en side: 3–5 timer (målt)
Change failure rate	hvor ofte brækker en ændring?	historik: 3 hændelser på 14 dage
MTTR	hvor hurtigt er vi tilbage?	1 t 38 min ved butiksnedbruddet 6/11
HTML og scripts	payload	75 KB / 27 scripts (forside)
Core Web Vitals	hastighed, respons, stabilitet	mål: LCP ≤2,5 s · INP ≤200 ms · CLS ≤0,1 ved p75 (feltdata)
Accessibility	kan flere bruge sitet?	mål: WCAG 2.2 AA; automatiseret + manuel kontrol
Kritisk brugerrejse	kan kunden nå sit mål?	mål: gennemført på mobil og desktop uden hjælp
Restore-tid	kan vi gendanne?	restore til staging gennemført og tidsmålt før launch
Fejl i loggen	500'er pr. uge	ingen uforklarede 5xx i kritisk rejse
Målehændelser	kan vi træffe beslutninger?	hver kritisk hændelse registreres én gang
Tilgængelighed	kan flere bruge sitet?	WCAG 2.2 AA er release-mål, ikke plugin-badge

De manglende performance-, accessibility-, restore- og event-baselines er huller i vores nuværende metode. De lukkes på et nyt site fra dag ét. Core Web Vitals er feltmål: brug p75 opdelt på mobil/desktop, når data findes. Før der er trafik nok, brug samme lab-opsætning før/efter som en foreløbig guardrail — og skriv udtrykkeligt, at lab-tallet ikke er felt-CWV.

5. Beslutningsmatrix — hvad skal det nye site have?

Site-type	Metode	Host	Begrundelse
Portfolio, brochure, 5–15 sider	block theme + Git + staging	managed WP eller delt med Git	ingen kode-tung drift
Webshop (som steamgears)	Git + staging + object cache	managed WP med SSH	checkout må ikke testes i live
Indholdssite, daglige publiceringer	Git til tema, staging til opdateringer	managed WP	plugin-opdateringer er risikoen
Medlemskab/betaling	Git + staging + testmiljø	managed WP eller VPS	fejl koster penge
App, flere kanaler	headless (Next.js + WPGraphQL)	separat frontend	kun når kanalerne kræver det
Legeprojekt under 500 kr./md.	delt hosting + Playground til demo	delt	accepter: ingen staging

Headless er ikke standarden. Det giver mening når indholdet skal bruges i en app, et andet site eller en kanal WordPress ikke selv kan vise — ellers betaler man for en ekstra frontend uden at få noget igen.

---

6. Playbook: nyt site i otte arbejdsfaser

Fase 0 er beslutningsporten; faserne 1–8 er de otte arbejdsfaser. Den vigtigste ændring fra den gamle arbejdsmåde er, at vi **beviser én komplet brugerrejse før vi masseproducerer sider eller funktioner**.

Første port: produktbrief og målinger

Før Blueprint eller tema skrives, udfyldes en kort brief. Den forebygger, at en færdig hjemmeside stadig er forkert for kunden.

Spørgsmål	Svar der skal være skrevet ned
Hvem er den primære bruger, og hvilket problem løser sitet?	Én målgruppe og ét konkret jobs-to-be-done
Hvad er den vigtigste handling?	køb, booking, kontakt, tilmelding eller læsning — én primær CTA
Hvordan ved vi, at løsningen er bedre?	1–3 måltal med baseline, ejer og målemetode
Hvad er MVP, og hvad venter?	must / should / later; ingen features uden brugerbehov
Hvem ejer fakta og godkendelser?	navngiv ejer af priser, jura, billeder, betaling og copy
Hvilke forudsætninger er usikre?	antagelse, hvordan den testes, og hvornår den afgør go/no-go
Hvad gør sitet med persondata?	felter, formål, opbevaring, adgang og sletning

Gate: ejer godkender mål, primary CTA, MVP og ejerafhængige fakta. Ukendte priser/jura/leveringsløfter bliver navngivne blokeringer — ikke gættet copy.

Prototypen skal bevise en hel rejse

Byg ikke først hele forsiden og håb, at resten passer. Prototypér én vigtig vej fra indgang til afsluttet opgave på mobil først, derefter desktop:

- **Webshop:** landingsside → produktside → variant → kurv → checkout → bekræftelse.
- **Lead/site:** landingsside → forstå tilbuddet → kontaktformular → succesbesked → bekræftelsesmail.

- **Indholdssite:** søgeindgang → svar → relateret indhold → relevant CTA.

Kør en kort, modereret **kvalitativ** opgavetest med 3–5 personer fra målgruppen. Det er en formativ fejlfindingsrunde — ikke statistisk markedsbevis eller en konverteringsprognose; se NN/g-kilden i §15. Notér hvor de stopper, hvad de misforstår, og om de fuldfører opgaven uden hjælp. Ret de kritiske og gentagne stopklodser, før hele sitet bygges. Book deltagerne tidligt: testens kalender-tid kommer oven i teamets aktive byggetid.

Beslutningsporte undervejs

- **Arkitekturport:** sitemap, indholdstyper, slugs, navigation og redirect-plan godkendes før copy-mængden vokser. URL'er er data: skift efter publicering kræver 301 og opdatering af interne links.
- **Designport:** `theme.json`, typografi, farver, spacing og 3–5 genbrugelige patterns fastlægges før alle skabeloner bygges. Én repræsentativ side godkendes på mobil + desktop.
- **Kvalitetsport:** accessibility, performance, sikkerhed, SEO, consent og måling køres på samme kritiske rejse — ikke kun på forsiden.
- **Go-live-port:** ingen åbne kritiske fejl i køb/kontakt/login, indholdsfakta godkendt af ejeren, backup gendannet til staging, rollback ejer og trigger kendt.

Fase	Arbejde	Tid	Gate
0 · port	Produktbrief, målgruppe, brugerproblem, CTA, KPI, MVP og ejerfakta	2–4 t	brief godkendt; usikre forhold har test/owner
1	Prototype i Playground (Blueprint i repo) + kritisk rejse	2–6 t	kvalitativ brugertest; gentagne stopklodser rettet
2	Host, staging, backup, PHP/WP-version	2–4 t	staging er noindex; restore prøvet
3	Repo, lokal opsætning, CI-lint	3–6 t	git push giver grøn kodekontrol
4	Block theme, theme.json, patterns, Woo-skabeloner	2–5 d	design godkendt på repræsentativ mobil/desktop
5	Indhold, metadata, schema, sitemap, redirects	1–3 d	URL-model låst; ingen udokumenterede facts
6	Staging-test: rejse, mails, formularer, a11y, performance	0,5–1 d	release-gates nedenfor bestået
7	Go-live: DNS, 301, sitemap, GSC, uptime	2–4 t	smoke-test efter DNS-skift + rollback-ejer
8	Drift: uge/måned/kvartal + dokumentation	løbende	7/30/90-dages-review og restore-øvelse

Port 0 er en beslutningsport uden for byggefaserne; **fase 1–8 er otte arbejdsfaser**.

Planlægningsestimat: 5–9 arbejdsdage for et lille site med eksisterende copy, **2–4 uger** hvis indholdet skal skrives. Det er et groft aktivt-teams-estimat, ikke en deadline-garanti; ejerafklaringer, rekruttering af testpersoner, host-setup og tredjepartsbetalinger kan forlænge kalenderen. Første sites gennemføres som pilot, og estimatet justeres ud fra faktisk tidsforbrug. Samme størrelsesorden som steamgears' deploy-roadmap, men fordelt på et site der ikke findes endnu.

Fase 1 — prototypen er kontrakten

Blueprint-filen er det første commit i repoet. Den giver tre ting: kunden ser sitetet før der er købt hosting, udvikleren får samme udgangspunkt hver gang, og demoen kan genskabes om et år. Læg den i `playground/blueprint.json` og skriv kommandoen i README.

Prototypen viser ikke bare hero og farver: den indeholder hele den kritiske rejse, den rigtige CTA, én fejltilstand, én tom tilstand og en succesbekræftelse. Test først den prototype med de målbrugere, der kan rekrutteres. Hvis de ikke forstår hvad siden tilbyder eller hvordan de fuldfører hovedopgaven, skal løsningen ændres før komponenter og copy skaleres op.

Fase 2 — hosten vælges på krav, ikke på pris

1. **SSH + WP-CLI** (ellers kan I ikke fejlfinde).
2. **Git-deploy** — indbygget eller via CI; staging deployes først, production-promote kræver accept.
3. **Staging** der kan klones, adgangsbeskyttes og sættes `noindex`; skriv hvad miljøet ikke matcher (PHP/cache).
4. **Backup** med restore til staging; kræv dokumenteret RPO/RTO fra hosten og mål restore selv.
5. **PHP/WP/Woo-version** er understøttet af tema og alle betalings-, fragt- og consent-plugins; lås versioner.
6. **EU-region, DPA og underleverandører** hvis målgruppen/kravene kræver det.
7. **Logs, cache purge og incident support** kan tilgås uden at kundens browser er eneste diagnosevej.

Sammenlign mindst to kandidater på de samme krav og samlet pris, herunder staging, backup og eventuelle migreringsgebyrer. Host-valg markeres som en beslutning med dato, ansvarlig og årsag — ikke et skjult gæt.

Fase 3 — repo og CI

Opret CI sammen med repoet, ikke som oprydning før launch. Hver ændring til production skal være en reviewbar PR/commit, grøn lint og deployet til staging. Production-promote er et separat, logget skridt. CI kører det der passer til koden (PHP syntax/style, theme/plugin checks, JavaScript/CSS build og automatiske smoke-tests); den må **ikke** deploye databasen eller `uploads/`.

Mindste repo der stadig er professionelt:

```
site/
  wp-content/themes/<tema>/      # block theme, theme.json, patterns
  wp-content/mu-plugins/         # små, uafhængige filer
  wp-content/plugins/<egen>/     # hvis eget plugin
  playground/blueprint.json      # demo og test
  .github/workflows/lint.yml     # php -l, phpcs, JS/CSS-build
  .github/workflows/deploy.yml   # staging, derefter manuel promote
  docs/                          # beslutninger, målinger, fælder
  README.md                      # lokal opsætning, kommandoer, ejerporte
```

Branch-model: `main` = produktion, `staging` = det der er på staging, `feature/*` = arbejde. Releases tagges (`v1.2.0`), så rollback er et tag og ikke en fil man leder efter.

Secrets: kun i hostens secret-store eller GitHub Secrets. Aldrig i `.env` der committes, aldrig i en option der kan læses udefra. En nøgle der har været eksponeret, tilbagekaldes **før** en ny oprettes.

Fase 4 — temaet

- `theme.json` ejer farver, typografi, spacing, `settings.viewport` og responsive varianter.
- **Patterns** til sektioner der gentages (hero, trust, produktgrid), så indhold ikke bliver copy-paste.
- **Skabeloner** til forside, arkiv, produkt, kurv, kasse og 404.
- Ingen hardkodet tekst i temaet; al redigerbar tekst er blokke eller patterns.
- Kontrast, fokusring, mobilnavigation og formularfelter testes, før designet godkendes — ikke efter.
- Designporten er en mobilbredde og desktopbredde på samme repræsentative side, ikke alle skærmstørrelser efter mavefornemmelse.

Fase 5 — indhold og SEO

Samme regler som deploy-playbook-2026-11-08.md §3.2: én H1, meta 120–160 tegn, canonical, ét OG-billede, schema der matcher den synlige tekst, og et sitemap der faktisk indeholder siden. Forskellen er, at det nu kan kontrolleres på **staging** i stedet for efter publicering.

Fase 6 — staging-testen og release-gates

Test den samme primære rejse, som blev godkendt i prototypen. Der publiceres ikke med kendte fejl, der forhindrer køb, kontakt, tilmelding eller den valgte hovedopgave.

Kvalitetsport	Krav før launch	Sådan bevises det
Brugerrejse	hovedopgaven fuldføres uden hjælp på mobil og desktop	modereret opgavetest; ingen blocker
Webfunktion	sidets kritiske formular/handling har success + fejltilstand	staging-kørsel med resultat/kvittering
Betaling (hvis webshop)	én godkendt og én afvist sandbox-betaling, ingen rigtig charge	payment sandbox + ordrenummer
Mails	ordrebekræftelse, fortrydelse/retur eller formkvittering	modtaget mail i testkonto
Accessibility	WCAG 2.2 AA som mål; tastatur, fokus, labels, kontrast og zoom kontrolleret	automatisk scan + manuel keyboardrunde; scanner alene er ikke bevis for AA
Performance	feltmål: LCP ≤2,5 s, INP ≤200 ms, CLS ≤0,1 ved p75 for mobil/desktop når data findes	CrUX/PageSpeed Insights; før data findes gemmes samme lab-test som baseline, markeret lab
SEO/links	1 H1, canonical, metadata, sitemap, alle kritiske links når rigtigt mål	crawl + følg redirect helt til målside, ikke kun 302
Analytics	hovedhændelse fyrer én gang og har korrekt værdi/valuta	network/event-debug; sammenlign med handlingen
Consent	afvisning starter ikke ikke-nødvendige scripts	test både reject og accept i frisk session
Sikkerhed	admin 2FA, mindst privilegier, secrets uden for Git, HTTPS	access review + secret scan
Backup/rollback	restore gennemført til staging; kode kan rulles til sidste release	kør øvelsen og mål tiden
Cache	kurv, checkout, login og personligt indhold ikke cachet	headers + brugerflow
Staging	password-beskyttet og noindex	HTTP/meta + Search Console-tilsyn efter launch

Performanceforbehold: Lighthouse/lab-tal er nyttige til regressioner, men erstatter ikke feltdata. Google anbefaler at vurdere Core Web Vitals på 75-percentilen, opdelt på mobil og desktop. Med for lidt trafik skrives status »feltdata utilstrækkelige« i stedet for at erklære målet bestået.

Go/no-go: stop ved fejl i betaling, persondata, adgang, hovedformular eller hovedrejse. Pynt, ekstra sider og ikke-kritiske animationer kan komme efter launch; de må ikke forsinke eller skjule en kritisk fejl.

Fase 7 — go-live og de første 30 dage

Før launch: lav TTL i god tid, 301 fra gamle URL'er, sitemap indsendt, GSC og analytics verificeret, uptime-overvågning tændt, og en smoke-test **efter** DNS-skiftet — ikke før. Skriv den sidste kendt-gode release, den der kan trykke rollback, og hvilke hændelser der udløser rollback (fx checkout-fejl, fatal fejl eller tab af ordre/formdata).

Efter launch:

- **Første døgn:** kontroller 5xx, checkout/formular, mail, betalinger, consent og deploy-status flere gange.
- **Dag 7:** gennemgå konverterings-/kontaktflow og fejl mod briefens baseline; ret den største dokumenterede friktion, ikke den nyeste idé.
- **Dag 30:** sammenlign KPI, Core Web Vitals feltdata hvis tilstrækkelige, uptime og supporthenvendelser mod baseline. Registrér beslutninger og nye værn i repoet.

- **Kvartalsvis:** restore-øvelse til staging og plugin-/adgangsgennemgang.

7. Startpakke — kommandoer

```
# 1. demo/prototype uden server
npx @wp-playground/cli@latest server --blueprint=playground/blueprint.json

# 2. lokalt miljø (vælg ét)
npx @wordpress/env start                    # kræver Docker + .wp-env.json
# eller: ddev config --project-type=wordpress && ddev start

# 3. hvis I vælger Composer-strukturen
composer create-project roots/bedrock my-site

# 4. tjek kode før push
find wp-content -name '*.php' -exec php -l {} +
```

En CI-deploy til staging er bevidst ikke skrevet færdig her: den afhænger af hosten, og en kopi-klistret `rsync`-kommando uden kendskab til stier og nøgler er farligere end ingen pipeline. Vælg host først, og skriv deploy-jobbet med hostens egen dokumentation ved siden af.

8. Hosting: kriterier og shortlist

Der er ingen valgt host endnu. Priser og planer ændrer sig, så de skal hentes hos den enkelte host, når beslutningen tages — ikke citeres fra denne guide.

Host	Git-deploy	SSH	Staging	Bemærkning (målt 20/10 2026)
WP Engine	Ja, GitPush + GitHub Action	Ja, gateway	Ja	wp-config.php og uploads kan ikke pushes
Pressable	Ja, GitHub-integration	Ja	Ja	integrationen er slået til for nye konti
WordPress.com	Ja, Business/Commerce	Ja, visse planer	Ja	GitHub Deployments
Kinsta	Nej, men Git + SSH	Ja	Ja, standard og premium	ingen push-to-deploy; CI eller pull
Raidboxes (DE)	Via SFTP/SSH og CI	Ja	Ja, gratis + add-on	staging deler ikke PHP/WP-version med live
Cloudways	Ja, auto-deploy via webhook	Ja	Ja	mere VPS end managed WP
Pantheon	Ja, Dev → Test → Live	Begrænset	Ja, indbygget	Test og Live er låste
Simply (nuværende)	Nej	Ikke brugt i driften	Nej	billig, kendt, ingen kode-deploy

Sådan vælges: skriv de seks krav fra §6 ned, og stryg enhver host der ikke opfylder dem. Vælg derefter på support, region og pris — ikke på logo. Når værten er valgt, er næste skridt miljøopsætningen i §9 — den afgør, om Git- og staging-metoden bliver sikker i praksis eller bare en dyr kopi af den gamle arbejdsmåde.

9. Server- og sandbox-forhold — den del der afgør, om metoden er sikker

Et site har tre miljøer, og hver har sit formål. Den dyre fejl er at gøre dem for ens: en sandkasse der ligner produktion så meget, at den kan trække penge, sende mail og ramme kundedata — eller en produktion der ligner en sandkasse, så der debugges live.

Miljø	Formål	Data	Må aldrig kunne
Lokal (wp-env, DDEV)	bygge, syntax, unit-tests	seed/fixtures	nå produktions-DB, sende rigtig mail
Sandbox/staging	accept, betalingstest, kvalitetsporte	kopi uden persondata eller syntetisk data	indekseres, trække penge, maile kunder, ramme live-webhooks
Produktion	kunder, ordrer, SEO	rigtig data	debug-output, testnøgler, eksperimenter

9.1 Produktionsserveren — baseline

Indstilling	Krav	Hvorfor
PHP	8.3–8.5, samme minor som er testet, opcache slået til	ellers tester man en anden runtime end kunden møder
WP_ENVIRONMENT_TYPE	production	plugins og tema må ikke tro, de er i staging
WP_DEBUG / WP_DEBUG_DISPLAY	false / false	ellers lækkes stier, SQL og data til besøgende
WP_DEBUG_LOG	kun hvis filen ligger uden for webroot eller er beskyttet	wp-content/debug.log kan ellers hentes offentligt
DISALLOW_FILE_EDIT	true	fjerner editor-vejen til en fatal fejl
HTTPS og secure cookies	hele sitet, HSTS når det er stabilt	login, kurv og kasse
Object cache	Redis/Memcached hvis hosten tilbyder det	DB-last og TTFB
Page cache-undtagelser	kurv, kasse, konto, login, REST	ellers lækkes personligt indhold
Cron	system-cron → wp cron event run --due-now	WP-Cron på et cachet site sover
Mails	SMTP/transaktionsudbyder + SPF, DKIM, DMARC	PHP-mail ender i spam eller blokeres
Backup	daglig DB + filer, offsite, 14–30 dages retention, restore testet	RPO/RTO skal være kendt, ikke håbet
Logs	PHP-fejl, slow-log, adgangslog bag auth, rotation	fejlfinding uden kundens browser
Rettigheder	deploy-bruger ≠ web-bruger, mindst privilegium, 2FA, SSH-nøgle med adskilt adgang	et kompromitteret plugin skal ikke eje filerne
Disk og inodes	kendt loft, logrotation og oprydning	et fuldt disk dækker over alle andre fejl

Mål det, antag det ikke. Et nyt site er først klart, når disse svar er hentet med kommando og gemt: hvilken PHP-version svarer, er debug slået fra, kører cron, kan en backup gendannes, og svarer `/wp-json/` uden at en cache eller WAF står i vejen. Kommandoerne står i §9.7.

9.2 Sandkassen — isoleret med vilje

Sandkassen er den farligste af de tre, fordi den har produktionens kode og ofte produktionens database. Den skal derfor lukkes på hvert et punkt hvor den kan gøre skade:

- **Eget domæne eller subdomæne**, `noindex` i metadata **og** `X-Robots-Tag`, plus HTTP-auth. En

indekseret staging er to sider der konkurrerer om de samme søgeord.

- **WP_ENVIRONMENT_TYPE=staging.** WordPress 7.1 slår pings fra uden for produktion, og plugins som betalingsudbydere skifter selv til test-adfærd ud fra miljøtypen.
- **Kun testnøgler og testkort.** WooCommerce anbefaler udtrykkeligt at teste på staging og aldrig på live, fordi en testordre kan udløse mails og forstyrre rigtige ordrer. Testordrer har **ingen visuel markering** i WooCommerce — slet dem efter testen.
- **Webhooks peges** på en sandbox-endpoint eller slås fra. Ellers kan staging oprette rigtige ordrer, abonnementer eller billetter i tredjeparten.
- **Mails fanges** i en mail-sink eller sendes kun til en intern testkonto. Et "disable emails"-plugin kan omgås af et SMTP-plugin, så verificér i loggen at intet gik ud.
- **Tracking og consent** bruger et separat property eller debug-tilstand. Staging skal ikke forurene kundens GA4, Ads eller Search Console.
- **Kanal-tokens fjernes**: nyhedsbrev, sociale medier, fragt og lager. En staging-kopi der publicerer eller bestiller varer er en hændelse, ikke en test.
- **Cron begrænses**: ellers publicerer staging kladder og tømmer kø-poster, der hører til live.
- **Cache og CDN er adskilt**: staging må ikke kunne purgre produktionens CDN.
- **Egen database og egne credentials.** Produktionens DB-bruger eller nøgler må ikke stå i staging-config.

9.3 Når staging-kopien skal tilbage

Retning	Regel	Grund
staging → live, kode	ja, via Git/release og samme gate som alt andet	det er den vej metoden findes for
staging → live, database	nej som standard; kun bevidst indholdsmigrering med backup, log og ejer-godkendelse	ordrer, kunder og abonnementer må aldrig overskrives
live → staging, database	kun anonymiseret eller syntetisk	persondata skal ikke ligge i et testmiljø
uploads	data, ikke kode — kopieres bevidst, ikke deployes	ellers overskrives nye billeder

WooPayments-fælden: en klon kan efterlade live-sitet "stuck in test mode", fordi indstillingerne `wcpay_account_data` og `wcpay_onboarding_test_mode` følger med databasen, og en klon kan aktivere Safe Mode, når to URL'er peger på samme konto. Læs deres migreringsdokumentation, før en butik klones — og kontroller indstillingerne efter promote.

9.4 Deploy-rækkefølgen — den optimale kæde

1. **PR** med lint, statisk analyse og tests. Ingen grøn pipeline, ingen merge.
2. **Automatisk deploy til staging** ved merge til staging-grenen — ikke til produktion.
3. **Smoke-test på staging**: health, login, kritisk rejse, testbetaling, mail-sink, `noindex`, cache-undtagelser.
4. **Manuel godkendelse** af et menneske, der ejer resultatet.
5. **Deploy kun kode** til produktion. Databaseændringer køres som et separat, logget skridt med backup først.
6. **Atomisk release** med forrige release bevaret, så rollback er et klik eller en symlink — ikke en gendannelse.
7. **Purge cache/CDN** og versionér assets, så kunderne ikke får gammel CSS med ny HTML.
8. **Smoke-test i produktion** og overvåg fejl de første 30 minutter.
9. **Log** commit, tag, ansvarlig og tidspunkt i repoet.

Lås og vindue: én deploy ad gangen (concurrency-gruppe), ingen deploy midt i en kampagne, og en skrevet regel for hvem der må trykke go. En deploy der fejler, standser linjen — den rulles tilbage eller rettes fremad med en ny release, aldrig med et manuelt indgreb i filerne.

GitHub-fælden (målt i dette repo): environments med required reviewers og wait timer er kun tilgængelige for **offentlige** repos på Free-planen. Bliver et repo gjort privat, ignoreres de konfigurerede beskyttelsesregler og environment-secrets. En "beskyttet" produktion kan altså se beskyttet ud uden at være det — brug branch protection og et manuelt `workflow_dispatch`-trin, eller en plan der understøtter det.

9.5 Paritet mellem miljøerne

Komponent	Skal matche produktion	Må afvige, hvis det er dokumenteret
PHP, WordPress, WooCommerce	ja	nej
Tema og plugins (kode)	ja	indhold må afvige
Webserver og rewrite-regler	så tæt som muligt	ja
Object cache og CDN	nej	ja — skriv hvad der ikke er testet
Mail, betaling, fragt	nej, med vilje	ja — det er hele pointen
Data	nej	ja — syntetisk eller anonymiseret

Paritet er ikke "alt er ens"; paritet er at **forskellene er kendte**. Skriv dem i repoets README, så næste person ikke tester noget andet end det, der bliver udgivet.

9.6 Sandbox for agenter og automatisering

- **Egne credentials pr. miljø**, scopede og roterbare. En produktionsnøgle i en sandkasse er en lækage med forsinkelse.
- **Ingen kundedata i en flygtig sandkasse.** Playground og lokale miljøer bruger seed-data.
- **Udgående trafik begrænses** hvor det er muligt, og logges. En agent skal ikke kunne sende mail eller kalde en betalings-API "fordi det virkede lokalt".
- **Mennesket ejer promote.** Automatik må deploye til staging; produktion kræver en godkendelse.
- **Grøn CI er ikke det samme som deployet.** Det er målt i dette repo: et grønt workflow-run kan se ud som en udrulning uden at være det.
- **Flygtige tokens dør** med sandkassen. Genskab dem med et script, og skriv hvordan i README.
- **Auditlog** over hvad automatikken ændrede, med commit eller release-tag som reference.

9.7 Kontrolkommandoer

```
# hvilket miljø tror WordPress, det er i?
wp eval 'echo wp_get_environment_type(), PHP_EOL;'

# er debug slået fra i produktion?
wp config get WP_DEBUG; wp config get WP_DEBUG_DISPLAY

# kører cron, og hvornår?
wp cron event list --fields=hook,next_run_relative

# hvilken release kører, og kan den rulles tilbage?
wp core version; git describe --tags --always

# svarer sitet uden at cache eller WAF forvirrer?
curl -sS -o /dev/null -w '%{http_code} %{time_total}\n' https://<domæne>/wp-json/

# er staging beskyttet mod indeksering og adgang?
curl -sSI https://<staging-domæne>/ | grep -iE 'x-robots-tag|www-authenticate'
```

9.8 Fælder i miljøopsætningen

Fælde	Hvad der sker	Regel
Staging med produktions-DB	kundedata lækker, og mails kan gå til rigtige kunder	anonymisér ved kopi, eller brug syntetisk data

Fælde	Hvad der sker	Regel
Test-mode hænger i live	butikken tager ikke imod rigtige betalinger	kontroller betalingsindstillingerne efter promote
Testordrer uden markering	de bliver behandlet, sendt eller talt med i statistik	slet dem efter testen
Webhooks peger på produktion	dobbelte ordrer eller abonnementer i tredjepart	egen sandbox-endpoint eller slået fra
Staging indekseret	to sider konkurrerer i søgeresultaterne	noindex + X-Robots-Tag + auth
Cron og pings kører	kladder publiceres, og søgemaskiner pinges fra et testmiljø	WP_ENVIRONMENT_TYPE=staging og begrænset cron
Debug slået til i produktion	stier, SQL og data vises til besøgende	WP_DEBUG=false, WP_DEBUG_DISPLAY=false
debug.log i webroot	logfilen kan hentes af alle	læg den uden for webroot eller beskyt den
Staging purger CDN	produktionens cache tømmes midt i trafik	adskilte nøgler pr. miljø
Miljø-gate der ikke findes	"beskyttet" deploy uden godkendelse	branch protection og manuelt promote-trin
PHP-version forskel	testet på 8.4, kørt på 8.2	lås versionen begge steder
Deploy uden smoke-test	grøn pipeline, hvid side	smoke-test efter hver promote

9.9 Måltal for miljøet

Måltal	Krav	Hvordan det bevises
Deploy-tid	under 5 min for kode	logget tid pr. release
Fejlede deploys	under 1 ud af 10	release-log
Rollback-tid	under 5 min	øvelse med stopur
Restore-tid (RTO)	kendt og målt	kvartalsvis restore til staging
Data-tab (RPO)	højst 24 timer	backupfrekvens og tidsstempel
Uautoriserede mails fra staging	0	mail-sink-log
Indekserede staging-URL'er	0	site:-søgning efter launch
Secrets ældre end 12 måneder	0	gennemgang af nøgler pr. miljø

Et miljø er først klart, når disse er sande: staging er utilgængeligt uden auth og `noindex`, kun testbetalinger kan gennemføres, ingen mail kan nå en kunde, cron og webhooks er begrænsede, produktion har debug slået fra, backup er gendannet én gang, og et menneske har godkendt den release der kører.

---

10. Hvis et eksisterende site skal flyttes

Flyt ikke for flytningens skyld. Flyt når der er en konkret gevinst: staging, kode-deploy, ydelse eller sikkerhed. Rækkefølgen er:

1. Frys indhold, tag fil- og DB-backup, og **test gendannelsen**.
2. Opret staging på den nye host, importér, og ret URL'er med `wp search-replace`.
3. Test kasse, mails, formularer, consent og 301'er på staging.
4. Sænk DNS-TTL, skift, og kør smoke-test.
5. Behold den gamle host i 30 dage.
6. Dokumentér hvad der blev ændret, og hvad der stadig er manuelt.

Risici der skal navngives: plugin-licenser bundet til domænet, serialiserede URL'er i databasen,

mail-flow der peger på den gamle host, WAF-regler der blokerer den nye, og betalingswebhooks der skal skiftes.

11. Fælder i den moderne metode

Den nye metode har sine egne fælder. De står her, så de ikke skal opdages på et nyt site.

Fælde	Hvad der sker	Regel
DB i Git	databasen committes eller overskrives	kode deployes, indhold migreres
Staging indekseret	to kopier konkurrerer i Google	noindex + password fra dag et
Composer vs. auto-update	core og plugins skiftes af to systemer	vælg ejer af opdateringer før deploy
Hostens plugin-forbud	et plugin kan ikke installeres	læs forbudslisten før arkitekturen vælges
Cache skjuler fejl	rettelsen ser ud som en ny fejl	purge før konklusion
Playground som produktion	ingen persistens, ingen betalinger	kun demo og test
theme.json uden review	én ændring flytter hele designet	styles i Git + style revisions
Deploy af uploads	billeder overskrives	uploads er data, ikke kode
Manglende restore-øvelse	backup findes, gendannelse gør ikke	kvartalsvis restore til staging

12. Tjekliste før launch

Gruppe	Punkt
Domæne	DNS, TTL, HTTPS, www → apex (eller omvendt), e-mail virker
Adgang	SSH nøgle, WP-CLI, staging-adgang, mindst to administratorer
Backup	daglig backup, off-site, restore testet
Sikkerhed	WAF, 2FA, ingen eksponerede nøgler, mindst mulige plugins
Miljøer	staging: auth, noindex, kun testbetaling, mail-sink · produktion: debug fra, cron kører, backup gendannet
Ydelse	HTML/script-budget målt, object cache, kurv/kasse ikke cachet
SEO	1 H1 pr. side, meta, canonical, sitemap, 301'er, OG-billeder
Butik (hvis relevant)	testordre, betaling, fragt, moms, mails, returformular
Jura (efter land/model)	CVR, adresse, telefon, fortrydelse, consent, GPSR
Måling	GA4 uden dubletter, GSC, uptime, fejl-log
Drift	deploy-rutine, rollback-rutine, kontaktvej, dokumentation

13. Forbedringer til vores fremgangsmåde — anbefaling til næste site

Disse ændringer anbefales som standard for nye sites oven på det vi allerede gør godt. De er tænkt til brug fra første brief; de kræver ikke at steamgears flyttes eller genbygges.

Prioritet	Forbedring	Ændring i arbejdsgangen	Hvilket svigt den forhindrer

Prioritet	Forbedring	Ændring i arbejdsgangen	Hvilket svigt den forhindrer
P0	Produktbrief før design	målgruppe, primary job, CTA, MVP, 1–3 KPI'er, fakta-ejer og antagelser skrives og godkendes i port 0	flotte sider uden mål eller ejerafklarede facts
P0	Én komplet brugerrejse først	prototype/test én kritisk rejse med 3–5 målbrugere, før massebygning	at opfinde hele sitet før man ser hvor folk stopper
P0	Staging + Git + rollback	alle kodeændringer via reviewbar commit → staging → accept → promote; DB/ordrer/uploads håndteres separat	direkte live-fejl, manglende diff og utryg rollback
P0	Release-porte	WCAG, kritisk rejse, betaling/formular, consent, analytics og backup indgår i go/no-go	»siden virker« betyder kun at forsiden åbner
P1	Målbar kvalitet	performance- og fejlbaselines måles før/efter; feltdata og lab-data mærkes hver for sig	falske hastighedsforbedringer og uklare claims
P1	Content model + URL'er før copy	indholdstyper, navigation, slug og redirectkort godkendes før produktion	masseindhold og dyre slug-/strukturændringer
P1	Mindre plugin-/feature-scope	hver plugin/feature skal have formål, ejer, dataadgang, vedligeholder og removal-plan	unødige afhængigheder, ekstra scripts og døde funktioner
P1	Launch er start på måling	24 timer / 7 dage / 30 dage + kvartalsvis restore review	ingen læring efter at sitet er live

Nyt standardforløb

1. **Brief og beslutningslog** — sæt succesmål, risici, MVP og ejerafhængige facts.
2. **Research og content model** — verificér konkurrenter/kilder, navngiv sidetyper, data og slugs.
3. **Prototype brugerrejsen** — Playground Blueprint + mobil/desktop, med fejl-, tom- og success-tilstand.
4. **Test og beslut** — kvalitativ test med målbrugere; log observationer, ret blocker, godkend scope.
5. **Byg repræsentativt** — temaets tokens/patterns + én komplet sidetype; review før bred genbrug.
6. **Byg resten i batches** — hver batch har staging-URL, owner, tests, acceptkriterium og rollback.
7. **Release-gates** — accessibility, performance, funktion, sikkerhed, SEO, consent og restore.
8. **Promote, smoke-test, mål igen** — kun ændret kode; indhold og kundeordrer migreres særskilt.
9. **Post-launch review** — 24 timer, 7 dage, 30 dage; opdatér KPI og dokumentér én læring pr. release.

Krav før vi kalder produktet færdigt

- Den primære bruger kan fuldføre den aftalte opgave på telefon og desktop.
- Ingen åbne kritiske fejl i checkout, betaling, formular, login, dataindsamling eller hovednavigation.
- Tilgængelighed er gennemgået med automated checks **og** manuel tastatur-/fokus-/formularrunde mod WCAG 2.2 AA.
- Performance er målt med dato, side, device, lab/felt og metode; ingen forbedring påstås uden før/efter-tal.
- Analytics registrerer den vigtigste konvertering én gang og med korrekt beløb/valuta, hvor det gælder.

- Backup er gendannet til staging, og rollback-trin + ansvarlig er kendt.
- Alle nødvendige fakta (pris, fragt, retur, identitet, samtykke) er ejet og godkendt — resten står som blokering, ikke som generisk placeholder.

---

14. Hvad vi tager med fra steamgears

Uanset host og metode skal det her følge med, fordi det er det der gjorde steamgears styrbart:

- **Et dokument per fælde**, med dato, symptom, årsag og værn.
- **Beregnet fremgang** — ingen procent uden regnestykke.
- **Maskinelle værn** i CI: form, drift, integritet, tests.
- **Måling før påstand**, også når svaret er "ikke målt".
- **Ejerporte** skrevet ned, så en blokering ikke bliver et gæt.
- **Sproget**: dansk i dokumentation og commits, engelsk på sitet.

---

15. Kilder

Kilde	Hvad den bruges til	Dato
Playground Handbook	Playground, Blueprints, lokal udvikling	2/11 2026
Playground CLI	npx @wp-playground/cli ... server	4/11 2026
Blueprints: Steps	JSON-skema for login, installTheme, setSiteOptions	14/10 2026
What's new for developers, august 2026	WordPress 7.1, iframed editor, settings.viewport, Site Manager API	8/2026
Theme Handbook: Tools and Setup	lokale miljøer: wp-env, DDEV, Studio	2023
Block themes	hvad et block theme er	—
WooCommerce: Using Blocks	block theme kræves til fuld store editing	2024
WooCommerce HPOS	HPOS er standard fra 8.2	—
Bedrock	Composer-struktur, .env, Git	—
Trellis deployments	zero-downtime deploy og rollback	—
15 hosts checked for Git deploy	hostmatrix: Git, SSH, staging	20/10 2026
Kinsta staging	staging, SSH, Git uden push-to-deploy	15/6 2026
Raidboxes staging	EU-host, staging og begrænsninger	28/7 2026
Core Web Vitals	p75-feltmål og tærskler: LCP 2,5 s · INP 200 ms · CLS 0,1	løbende
WCAG 2.2	anbefalet tilgængelighedsmål for nye sites	12/12 2024
WordPress Accessibility Coding Standards	WordPress anbefaler WCAG 2.2 AA for nyt/opdateret kode	14/10 2026
WordPress brute-force guidance	2FA, mindst privilegium, HTTPS, testede backups	14/10 2026
NN/g: 5 participants, qualitative vs. quantitative	lille brugertest er formativ fejlfinding, ikke kvantitativ dokumentation	11/7 2021
wp_get_environment_type()	miljøtyper: local, development, staging, production; standard er produktion	—

Kilde	Hvad den bruges til	Dato
Pings slås fra uden for produktion	WordPress 7.1 slår pingbacks fra i ikke-produktionsmiljøer	7.1
Debugging in WordPress	WP_DEBUG, WP_DEBUG_DISPLAY, WP_DEBUG_LOG; debug frarådes på live	14/10 2026
WooCommerce: testing orders	test kun på staging; testordrer har ingen markering og kan udløse mails	8/5 2024
WooPayments: site migrations	Safe Mode ved klon; "stuck in test mode" via wcpay_account_data og wcpay_onboarding_test_mode	9/11 2023
WooCommerce: skift af tema	bevar live-ordrer; overskriv ikke live-DB med en ældre staging-kopi	—
Stripe: testing og sandboxes	sandbox er isoleret; test- og live-nøgler og webhook-secrets er adskilte	—
GitHub: deployments og environments	required reviewers, wait timer og environment-secrets; på Free kun for offentlige repos	—

Ikke undersøgt her: konkrete priser/planer, dansk databehandleraftale pr. host, Lighthouse-tal for et nyt site, og hvilke af §9's miljøkrav den enkelte host faktisk leverer (object cache, system-cron, SMTP, separat staging-DB, miljøvariabler). Det skal afklares og måles pr. host, når værten er shortlistet — ikke antages ud fra denne guide.